Teoria Współbieżności

Kajetan Frątczak 04.12.2025 r.

Laboratorium 08

Asynchroniczne wykonanie zadań w puli wątków przy użyciu wzorców Executor i Future

1. **Treść zadań**

   Zadanie 1

   1. Proszę zaimplementować przy użyciu Executor i Future program wykonujący obliczanie zbioru Mandelbrota w puli wątków. Jako podstawę implementacji proszę wykorzystać kod w Javie.
   2. Proszę przetestować szybkość działania programu w zależności od implementacji Executora i jego parametrów (np. liczba wątków w puli). Czas obliczeń można zwiększać manipulując parametrami problemu, np. liczbą iteracji (MAX_ITER).

2. **Argumentacja rozwiązania**

   Zadanie 1

   W celu rozwiązania problemu współbieżnego generowania fraktala Mandelbrota zastosowałem wzorce projektowe Executor i Future. Kluczowe decyzje projektowe: dekompozycja zadania (obraz podzielono na niezależne wiersze, gdzie każdy wiersz jest osobnym zadaniem Callable<Void> - taki podział zapewnia: brak współdzielonym zasobów między zadaniami, równomierny podział pracy, prostą synchronizację poprzez Future.get()), wybór typów Executor - przetestowałem 5 implementacji: SingleThreadExecutor - benchmark sekwencyjny, FixedThreadPool - stała pula wątków, CachedThreadPool - dynamiczna pula, tworzy nowe wątki w razie potrzeby, ScheduledThreadPool - zaprojektowany dla zadań cyklicznych, WorkStealingPool - wykorzystuje algorytm work-stealing. Taki model minimalizuje synchronizację i pozwala w pełni wykorzystać zasoby CPU.

3. **Rozwiązanie**

   Zadanie 1

```java
package pl.edu.agh.to;
import java.awt.Graphics;
import java.awt.image.BufferedImage;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import javax.swing.JFrame;

public class MandelbrotExecutor extends JFrame {

   private final int MAX_ITER;
   private final double ZOOM = 150;
```

```java
private BufferedImage I;
private int width = 800;
private int height = 600;

// dodałem maxIter, żebym mógł modyfikować pod testy
public MandelbrotExecutor(int maxIter) {
    super("Mandelbrot Set - Executor Service");
    this.MAX_ITER = maxIter;
    setBounds(100, 100, width, height);
    setResizable(false);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    I = new BufferedImage(getWidth(), getHeight(), BufferedImage.TYPE_INT_RGB);
}

// Zadanie obliczające jeden wiersz obrazu
private class RowTask implements Callable<Void> {
    private int y;

    public RowTask(int y) {
        this.y = y;
    }

    @Override
    public Void call(){
        double zx, zy, cX, cY, tmp;
        for (int x = 0; x < width; x++) {
            zx = zy = 0;
            cX = (x - 400) / ZOOM;
            cY = (y - 300) / ZOOM;
            int iter = MAX_ITER;
            while (zx * zx + zy * zy < 4 && iter > 0) {
                tmp = zx * zx - zy * zy + cX;
                zy = 2.0 * zx * zy + cY;
                zx = tmp;
                iter--;
            }
            // Zapis do bufora (wiersze są rozłączne, więc tak mogę zrobić)
            I.setRGB(x, y, iter | (iter << 8));
        }
        return null;
    }
}

public void runTest(ExecutorService pool, String poolName) {
    // Reset obrazu
    I = new  BufferedImage(getWidth(), getHeight(), BufferedImage.TYPE_INT_RGB);

    long startTime = System.currentTimeMillis();
    List<Future<Void>> futures = new ArrayList<>();

    // Zlecanie zadań - 1 zadanie = 1 wiersz
    for(int y = 0; y < height; y++){
        futures.add(pool.submit(new RowTask(y)));
    }

    // Oczekiwanie na zakończenie wszystkich zadań
    for(Future<Void> future : futures){
        try {
            future.get();
        } catch (InterruptedException | ExecutionException e){
            e.printStackTrace();
        }
    }

    long endTime = System.currentTimeMillis();
    System.out.println("Pool: " +  poolName);
    System.out.println("Total time taken: " + (endTime - startTime) + " ms");
```

```java
        pool.shutdown();
        repaint();
    }

    @Override
    public void paint(Graphics g) {
        g.drawImage(I, 0, 0, this);
    }

    public static void main(String[] args) throws InterruptedException {

        // Scenariusze testowe
        int[] threadCounts = {1, 2, 5, 10, 15, 20};
        int[] maxIters = {100, 250, 500, 1000, 5000};

        System.out.println("=== Mandelbrot Set Performance Test ===");

        for (int maxIter : maxIters) {
            System.out.println("Max iterations: " + maxIter);

            // Nowy executor dla danej liczby iteracji
            MandelbrotExecutor executor = new MandelbrotExecutor(maxIter);
            //executor.setVisible(true);

            for (int threads : threadCounts) {
                System.out.println();
                System.out.println("Thread count: " + threads);
                // 1. Single Thread
                if (threads == 1) {
                    executor.runTest(Executors.newSingleThreadExecutor(), "SingleThread");
                    Thread.sleep(1000);
                }

                // 2. Fixed Thread Pool
                executor.runTest(Executors.newFixedThreadPool(threads), "FixedThreadPool");
                Thread.sleep(1000);

                // 3. Cached Thread Pool
                executor.runTest(Executors.newCachedThreadPool(), "CachedThreadPool");
                Thread.sleep(1000);

                // 4. Scheduled Thread Pool
                executor.runTest(Executors.newScheduledThreadPool(threads), "ScheduledThreadPool");
                Thread.sleep(1000);

                // 5. Work Stealing Thread Pool
                executor.runTest(Executors.newWorkStealingPool(threads), "WorkStealingThreadPool");
                Thread.sleep(1000);
            }
        }

    }
}
```

4. Wykresy, tabele, wyniki

MAX_ITER = 100

Thread count	SingleThread	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	WorkStealing
1	36 ms	53 ms	64 ms	59 ms	75 ms
2	—	28 ms	81 ms	34 ms	37 ms
5	—	21 ms	31 ms	20 ms	22 ms
10	—	14 ms	12 ms	56 ms	12 ms
15	—	21 ms	8 ms	24 ms	22 ms
20	—	23 ms	25 ms	20 ms	23 ms

MAX_ITER = 250

Thread count	SingleThread	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	WorkStealing
1	79 ms	82 ms	29 ms	61 ms	89 ms
2	—	44 ms	28 ms	46 ms	37 ms
5	—	30 ms	12 ms	40 ms	34 ms
10	—	19 ms	24 ms	18 ms	26 ms
15	—	25 ms	29 ms	29 ms	27 ms
20	—	17 ms	75 ms	11 ms	30 ms

MAX_ITER = 500

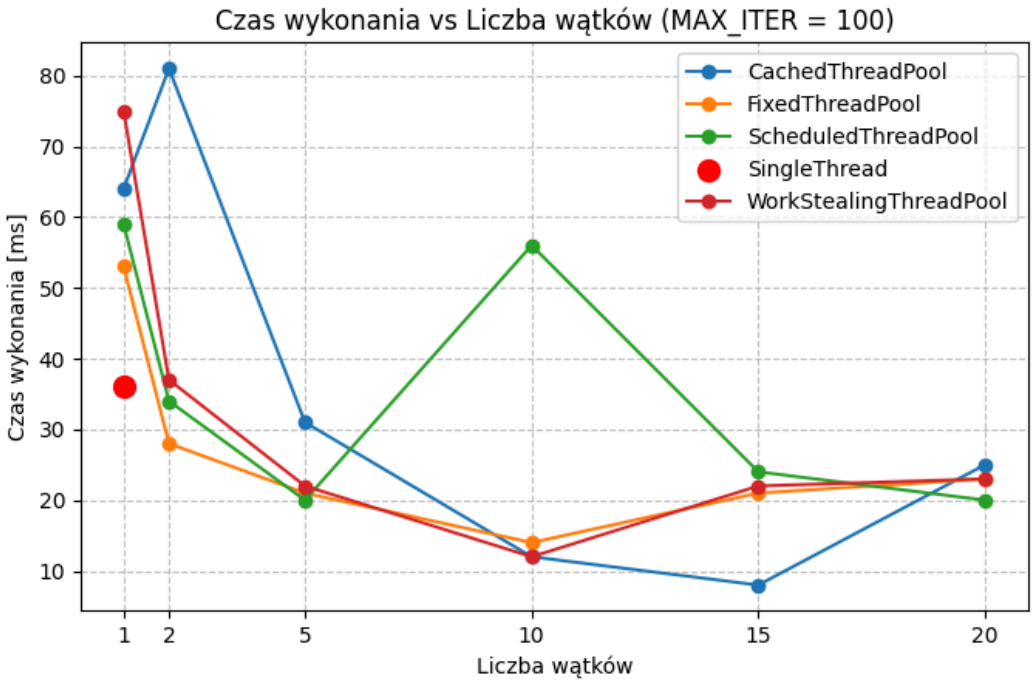
Thread count	SingleThread	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	WorkStealing
1	110 ms	91 ms	18 ms	90 ms	149 ms
2	—	54 ms	36 ms	75 ms	110 ms
5	—	50 ms	37 ms	47 ms	49 ms
10	—	34 ms	38 ms	36 ms	35 ms
15	—	13 ms	85 ms	30 ms	38 ms
20	—	36 ms	38 ms	33 ms	37 ms

MAX_ITER = 1000

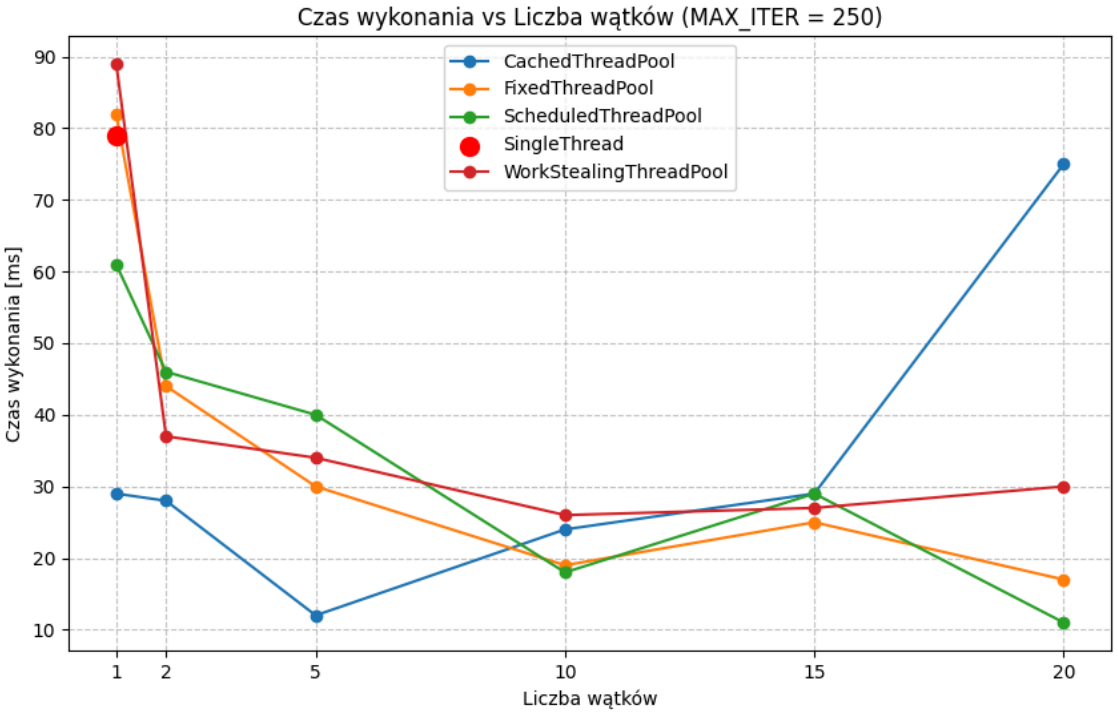
Thread count	SingleThread	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	WorkStealing
1	160 ms	183 ms	91 ms	174 ms	209 ms
2	—	79 ms	40 ms	83 ms	79 ms
5	—	62 ms	40 ms	59 ms	69 ms
10	—	31 ms	54 ms	52 ms	58 ms
15	—	51 ms	47 ms	53 ms	51 ms
20	—	56 ms	92 ms	51 ms	55 ms

MAX_ITER = 5000

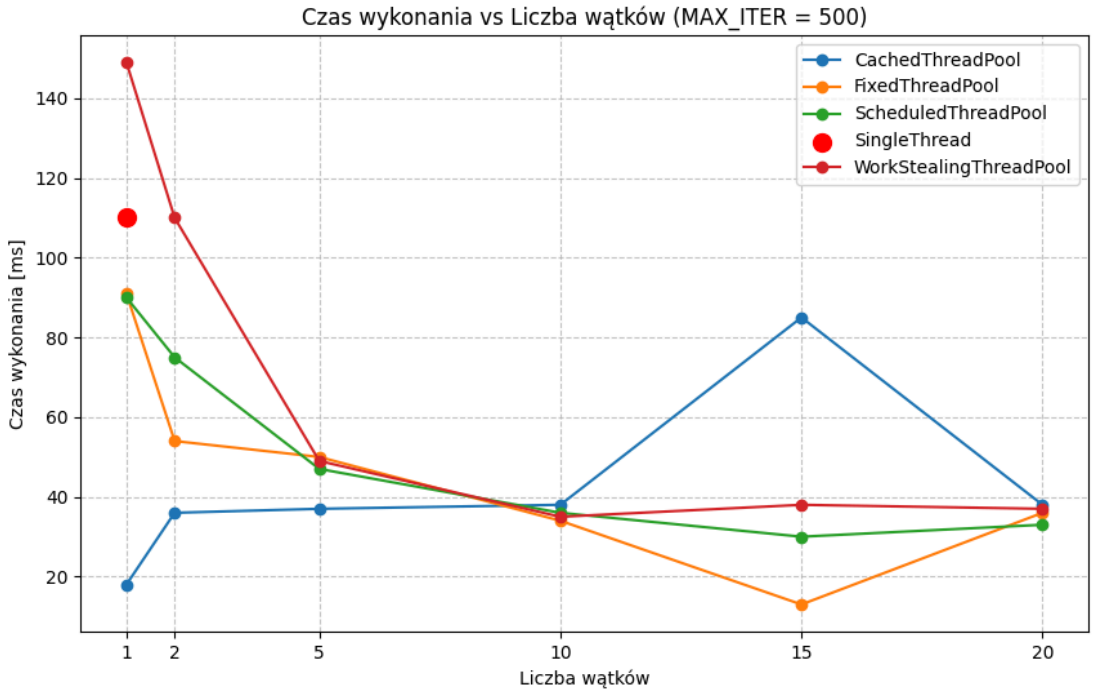
Thread count	SingleThread	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	WorkStealing
1	690 ms	631 ms	111 ms	907 ms	758 ms
2	—	303 ms	123 ms	311 ms	301 ms
5	—	148 ms	116 ms	158 ms	147 ms
10	—	119 ms	127 ms	76 ms	75 ms
15	—	115 ms	103 ms	123 ms	124 ms
20	—	73 ms	136 ms	103 ms	121 ms



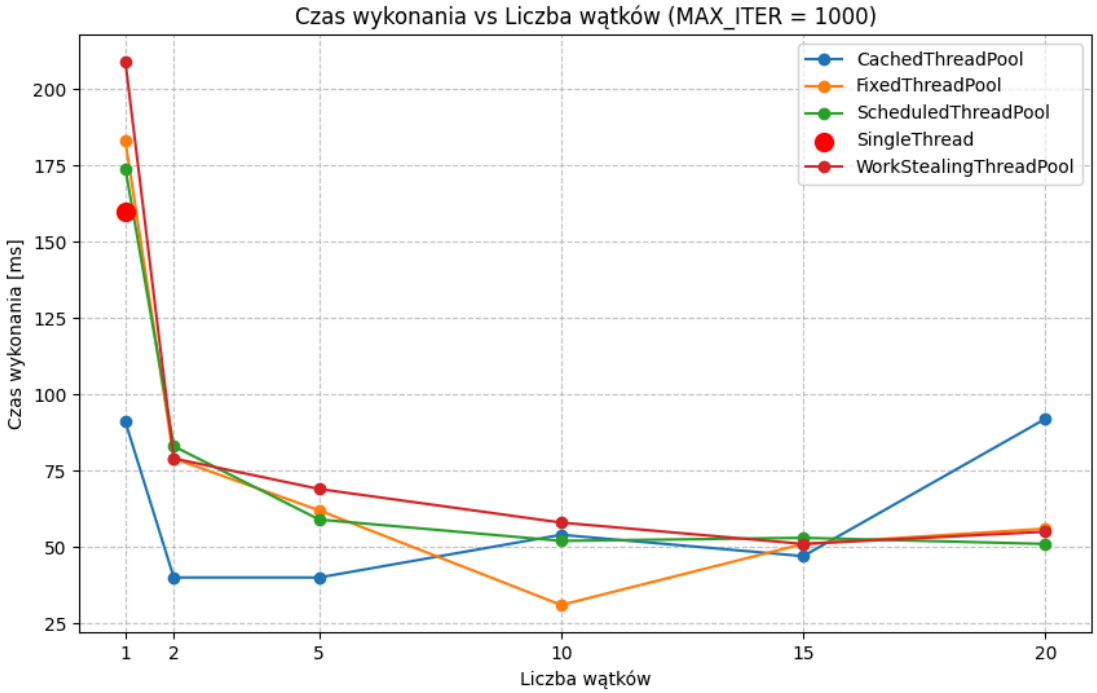
Rysunek 1: Wykres porównania czasów dla MAX_ITER = 100



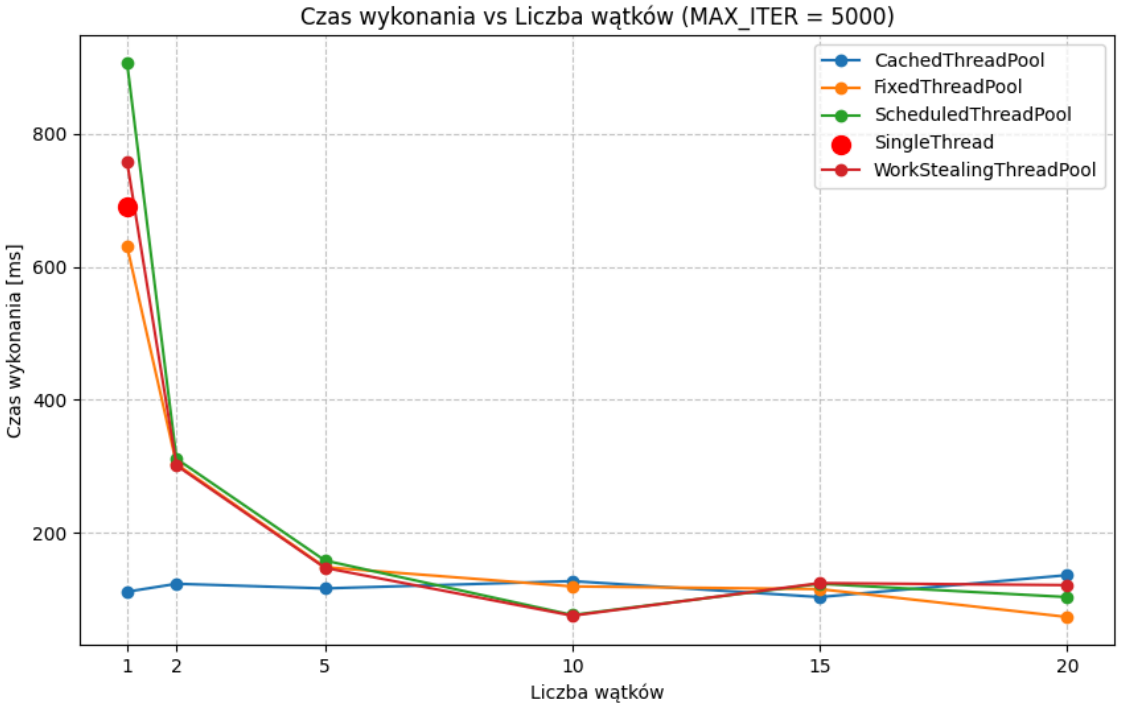
Rysunek 2: Wykres porównania czasów dla MAX_ITER = 250



Rysunek 3: Wykres porównania czasów dla MAX_ITER = 500



Rysunek 4: Wykres porównania czasów dla MAX_ITER = 1000



Rysunek 5: Wykres porównania czasów dla MAX_ITER = 5000

5. Wnioski

- Przy małych wartościach MAX_ITER widzimy niewielką przewagę wielowątkowości, zaś dla większych MAX_ITER przewaga ta jest znacząca - wielowątkowość znacznie przyspiesza obliczenia.
- FixedThreadPool - najbardziej przewidywalna wydajność, najlepsza dla większej liczby wątków.
- CachedThreadPool - nieoczekiwanie słaba wydajność i nieprzewidywalność.
- WorkStealingPool - bardzo dobra wydajność, działa najlepiej przy dużych obciążeniach.
- ScheduledThreadPool - wydajność porównywalna z FixedThreadPool, ale duży narzut w związku z mechanizmem szeregowania.
- Anomalie w wynikach jakie możemy zauważyć szczególnie przy mniejszych wartościach MAX_ITER mogą wynikać z błędów pomiarowych (pomiar robiłem tylko raz), narzutów JVM, braku rozgrzewki JVM oraz względnie krótkiego czasu wykonania, na którym jakieś anomalie są bardziej zauważalne. Przy MAX_ITER = 5000 wyniki są już stabilne.

6. Bibliografia

https://rosettacode.org/wiki/Mandelbrot_set#Java